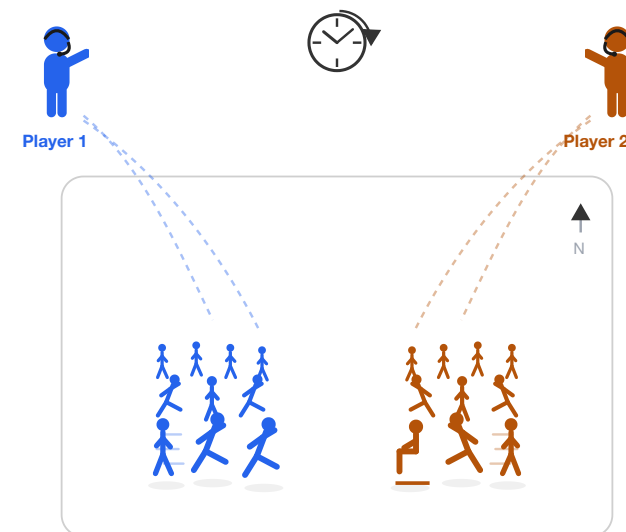


Real-Time Crowd Commander

Natural-language command of agent crowds in strategy games, under real-time compute budgets



The interface: command a battle by voice, watching and listening as it unfolds.



The endgame: several players each command their own crowd, in one real-time world.

A project proposal grown from Yubo Huang's interest and insight, under the guidance of Dr. Enmao Diao.

June 2026

1.1 The Problem and the Vision

- In a strategy game, the **fun is the strategy**: where to expand, when to attack, how to read the opponent.
- It is buried under **frantic clicks**: hunting icons, drag-selecting, hundreds of actions a minute.
- A player should command like a **real commander**: watch, listen, **speak the orders**, press a key only now and then.
- The subordinates execute. That interface could be the next strategy game.



Figure 1: the labour the interface forces. A few StarCraft II tasks (move here, gather there, build these); the strategy is buried under hundreds of manual actions a minute.

Still: PySC2 video (DeepMind)

1.2 The Gap: Efficiency, Untested at Game Speed

- The interface shipped once (**EndWar, 2008**), but on a 70-word grammar. Language models lifted that limit; they still **cannot act at game pace**.
- LLMs already play RTS through text (**TextStarCraft II**, NeurIPS 2024), but only by **pausing the clock**.
- The methods that would fix that, above all **KV-cache eviction**, are scored on perplexity, **never in a live game** where evicting the wrong memory loses the match.
- A streaming game is the natural stress test, and **nobody has run it** (the LLM Game Agents Survey, CSUR 2026, names it a core open challenge).

| **How much does command cost, in latency and memory, at game speed, and how do we drive it down?**

2 Preliminaries

The background this work builds on:

Area	What it gives us
Reinforcement learning	A game is a Markov decision process, win rate is the reward, recalling a scouted tech switch is credit assignment. We need RL <i>literacy</i> to read the field, not RL training.
Efficient LLM inference	KV-cache eviction (H2O, SnapKV, StreamingLLM, <u>OBCache</u>), structured pruning, quantization, distillation: the methods this work puts to the test.
Vision-language-action models	<u>π_0</u> : a slow vision-language backbone driving a fast action expert, trained by imitation. A model for splitting a slow strategic brain from fast executors.
The StarCraft II agent stack	<u>PySC2</u> , <u>TextStarCraft II</u> , LLM-PySC2: the environment we inherit rather than rebuild.
Tokenization beyond text	Byte-pair encoding and <u>graph tokenization</u> (<u>ICLR 2026</u>): the basis for a learned game-state tokenizer.

3.1 The Three Jobs

The system needs three capabilities: separate lines of work that eventually run together.

Job	What it does
Decide	An LLM commander reads the game-state stream and issues the orders, under hard latency and memory budgets
Foresee	A compact world model answers "what if" before you commit: if the army pushes now, does the fight win?
Embody	Units carry out the orders with model-generated motion, at crowd scale, on one GPU

This proposal is about the first job, **Decide**. Foresee and Embody come later.

3.2 The Plan: Three Phases

The jobs are the *parts*; the phases are the *order*. One shared **harness** runs under all of them (command protocol, unpausable clock, deadlines, metric logging), so the engineering transfers upward.

Phase	Focus	What happens
1	Benchmarks	The command arena (warm-up), then StarCraft II with the clock unpaused.
2	Methods	Attack whatever the benchmarks expose: game-aware eviction, a learned tokenizer, distilled commanders.
3	Real interface	The human (voice), then humans plural (multiplayer); grow the Foresee and Embodiment jobs.

Environments grow with the phases: **a toy room** → **StarCraft II** → **multiplayer** → **a full game**. The end-state game is the north star, not a deliverable.

4.1 The Command Arena

- Colour-tagged agents in a room; each moves in **one of four directions** on command.
- One command is trivial; the test is the **stream**: many, fast.
- Harder with **rate**, **compositional orders** ("everyone but the yellow one"), and **memory** ("the one I moved west").
- Measured: **grounding accuracy**, **command-to-action latency**, **deadline misses**.

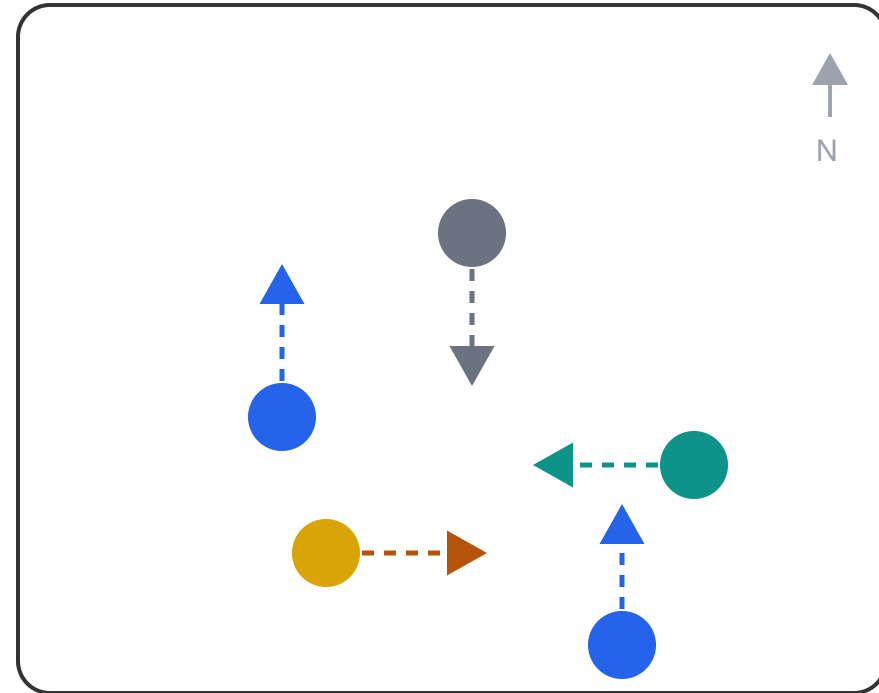


Figure 2: the command arena. Colour-tagged agents, each moving in one of four directions on command.

4.2 Then StarCraft II

- Once the arena works, the same harness moves to **StarCraft II with the clock unpaused**: real strategic depth, a real opponent.
- There the efficiency methods (which cache policy, how big a model, how to encode the state) are scored by **win rate under a latency and memory budget**.
- Concrete benchmark design waits until the arena is built.

5 What This Produces

The game is the north star; the papers are the milestones. Each answers a question its community already cares about, beyond the game.

Paper	Phase	The question	Who cares
Command-arena benchmark	1	How does grounding degrade as command rate rises?	Real-time agents, interactive systems
Real-time commander benchmark	1	Which efficiency methods survive a closed-loop game clock?	KV-cache and pruning researchers
Game-state tokenizer	1 to 2	Does compact tokenization extend to entity and event streams?	Tokenization beyond text
Competitive-efficiency study	3	Does a fast small commander beat a slow large one?	Inference efficiency, agents
Crowd motion under budget	3	Can language-commanded crowds move in real time on one GPU?	Motion generation, graphics